AIWA Yellow Paper

Causal Coordination and Local Value Accrual for Partition-Tolerant Networks

Version 2.0 — formal specification, reference implementation

Abstract

AIWA is a value-accrual and causal-coordination primitive for networks under arbitrary delay, intermittent connectivity, and unbounded partition. It separates two properties conventionally coupled in distributed ledgers:

Coordination. A deterministic causal layer over authenticated events, rooted in a common genesis. Domains operate while disconnected; reconciliation is deferred, not required.

Accrual. Local, unconditional creation of value, gated at genesis by an external commitment (§8) and thereafter driven by real sequential computation (§6) — never by a shared clock.

Progression → claimable value Conservation → ownership Mirror → verifiable history

No component requires a globally synchronized state.

1 System model

A domain is an operational environment holding one or more identities and local state. An identity is a keypair. Communication between domains is continuous, intermittent, delayed, asymmetric, or absent, for arbitrary duration.

Assumed: collision-resistant hashing, EUF-CMA-secure signatures, deterministic serialization. **Not assumed:** a synchronized wall clock, continuous access to any external chain, a global consensus quorum, a globally replicated state.

2 Identity

$$\text{domain}(i) = \text{SHA-256}(\text{pk}_i)$$

Full 256-bit digest, hex-encoded, untruncated. The private key authorizes state transitions; no other binding — device, IP, location — is part of identity.

3 Event DAG

State transitions are signed events referencing causal parents, forming a DAG. A participant need not hold every event, only what is relevant to its own state and observed relationships. $G_1 \cup G_2$, for two independently-grown DAGs, is a commutative union.

3.1 Content addressing

$$\text{id}(e) = \text{SHA-256}\big(\text{JSON}(\text{canon}(\{\text{parents} : \text{sort}(e.\text{parents}), \text{payload} : e.\text{payload}\}))\big)$$

where $\text{canon}(v)$ is defined recursively: for an array, canon applied element-wise, order preserved; for an object, keys sorted lexicographically and canon applied to each value under its sorted key; otherwise, v

unchanged. This is the one real specification every other section's own use of "the same event" depends on — Mirror's receivedFrom (§4), the relative rate witness's sourceEventId (§14), all reference id(e) under this exact definition. An independent implementation that canonicalizes differently — different key-sort locale, different JSON number formatting, different whitespace — produces a different id for identical semantic content, silently breaking interoperability without any signature failing. Verified concretely alongside §6's own cross-runtime check (`interop/rust-vdf/`): a real, independent Rust implementation of canon and id(·) matches the real JS output byte-for-byte for real, non-trivial test events (nested payloads, multiple unordered parents).

4 Mirror

$M_D \in \{\texttt{empty}, \texttt{full}\}$: a domain D's own signed, per-epoch commitment to what it has observed of another domain, never a replica of the observed state.

Reception monotonicity. For domain D observing X: $\text{seen}_D(X, e_{i+1}) \geq \text{seen}_D(X, e_i)$ for successive real commitments e_i. A claim of having seen less than previously committed is rejected.

Mirror does not establish that two domains are distinct real-world actors, nor rule out a coalition fabricating consistent history together at real cost. It is evidence about the structure of observed history — not an identity oracle.

5 Progression

$$\text{epoch}_D(n+1) = \text{epoch}_D(n) + 1$$

valid only if causally chained to D's own last accepted transition and carrying a real sequential proof (§6). Progression is local: epoch_A and epoch_B are never directly comparable.

6 Sequential proof

$$h_0 = \text{SHA-256}(\text{seed}), \qquad h_i = \text{SHA-256}(h_{i-1})$$

seeded from $(\text{domain}, \text{output}_{n-1})$. Symmetric — verification cost equals production cost — unlike an asymmetric VDF (Wesolowski, Pietrzak); what is unconditional is that the chain imposes VDF_ITERATIONS genuinely dependent, sequential steps — no amount of hardware lets a later step be computed before an earlier one. How much *real time* those steps take is not unconditional: real, independent SHA-256 benchmarks show roughly a 2–4× spread between common hardware with and without dedicated SHA acceleration (Intel/AMD SHA-NI, ARM's own SHA2 extensions) — a real, bounded, measured gap, never claimed to be hardware-invariant, and structurally far smaller than a parallelizable proof-of-work's own gap precisely because sequentiality (above) rules out scaling by adding more hardware in parallel.

Locality. Computation of h_i occurs on exactly one real device at a time — never distributed, never assisted by other domains. Mirror and live sync (§4, §13) carry only already-computed results. Identity is the keypair (§13.1); the computing device may change without discontinuity in epoch_D.

Rate, not calendar time. A fixed-length chain (VDF_ITERATIONS = 12000 in this deployment) measures $\sim$ 256ms of real computation — a floor, not a symmetric clock: real hardware differs across domains, so epoch_D is a true measure of D's own sequential work, correlated with but not equal to elapsed real time for any other domain. Comparing raw epoch counts across domains, or adjusting one domain's earned value against another's, would either discount genuine work under weaker hardware or credit idle time for merely reconnecting (§13).

6.1 Asymmetric verification (Wesolowski)

The symmetric chain (above) costs a verifier exactly what it cost the prover — real, but expensive to recheck at scale. A real Wesolowski VDF, over $\mathbb{Z}_N^*$ for a 2048-bit RSA modulus N of unknown factorization (the real, published RSA-2048 challenge number, cross-checked digit-for-digit against three independent sources — Wikipedia, an encyclopedia mirror, a math blog), gives verification cost independent of the real iteration count T:

$$y = x^{2^T} \bmod N \qquad \text{(evaluation: } T \text{ real, sequential squarings)}$$

$$\ell = \text{HashToPrime}(x, T, y) \qquad \pi = x^{\lfloor 2^T/\ell \rfloor} \bmod N \qquad \text{(proof: a real, separate } T\text{-step pass)}$$

$$r = 2^T \bmod \ell \qquad \text{Verify: } \pi^\ell \cdot x^r \stackrel{?}{=} y \pmod N$$

Real prover cost is on the order of $2T$ modular multiplications (evaluation's own T plus the proof's own T — the proof does not free-ride on evaluation's work, since ℓ depends on the real y). Real verifier cost is $O(\log T)$ — the entire point: unconditionally cheap regardless of how large T grows, verified concretely (a fabricated y, or a proof computed against a wrong ℓ, is rejected; verification time does not grow with T). ℓ is derived deterministically from (x, T, y) via a real hash-to-prime — never accepted as prover-supplied input, closing the one real freedom a dishonest prover would otherwise have.

7 Accrual

$$R(S, t, A) = \frac{S \cdot t^\alpha}{[\beta \ln A + \ln(1 + C/A^\beta)]^\gamma}$$

Symbol	Meaning
S	committed capital (§8, cumulative)
t	epochs since D's own last economic action (burn or claim); resets on each
A	D's own total progression age; never resets
α, β, γ, C	deployment parameters

Invariant. t and A are derived exclusively from D's own verified progression state at query time — never accepted from an event payload. A caller-supplied reference epoch would permit unbounded t.

t resetting on claim encodes patience *since last action*, not since genesis; A, unreset, is a maturity denominator independent of claiming frequency.

Computable form: $\beta \ln A + \ln(1 + C/A^\beta) \equiv \ln(A^\beta + C)$, represented in the left form to bound overflow for large A.

8 Genesis Commitment

$$b_D = \sum_{\text{valid burns}} \text{lamports}_i$$

Activation requires an irreversible SOL burn to Solana's incinerator address, verified from a finalized transaction record. Cumulative across every valid burn, each independently checked against the deployment's own churn cost curve at its own slot — a later commitment is never grandfathered at an earlier, lower requirement.

R is linear in S: absent a genesis cost, splitting capital across identities would not reduce total accrual. A per-identity activation cost makes churn strictly costlier, not free.

8.1 Whether churn pays

Existence of a real cost is not the same claim as sufficiency. For a real span of N epochs and committed capital S:

$$\text{stay}(N) = R(S, N, N) - \text{cost}(0)$$

$$\text{churn}(N, k) = \left\lfloor \frac{N}{k} \right\rfloor \cdot [R(S, k, k) - \text{cost(slot at cycle start)}]$$

comparing one domain that commits once and matures for the full span against one that restarts every k epochs, repeatedly re-entering at low A where R's own denominator is smallest. Verified concretely: with zero real cost, churn wins outright (12.31 against 5.67 over identical parameters); with a real, deliberately-chosen cost curve, churn nets negative (-27.69) while staying nets positive (3.67) — the identical R, the only real difference being cost$(\cdot)$'s own magnitude. `findMostProfitableChurnInterval` sweeps k over a real candidate range to find an attacker's own real best case, rather than checking one interval and declaring victory. This is a real calculator, computed per deployment's own chosen $(\alpha, \beta, \gamma, C)$ and cost curve — never a general proof that any given tuple is safe.

External dependency. Broadcasting a burn, or any further Solana-side action, requires reaching a centralized, Earth-hosted RPC endpoint over real internet — the one exception to this document's own no-shared-infrastructure principle. Once activated, epoch_D requires no further contact with Solana or Earth.

9 Conservation

A claim is a tuple $(\text{id}, \text{amount}, \text{owner}, \text{status} \in \{\text{active}, \text{inactive}\})$. Neither operation below depends on continued access to progression or the activation chain.

Split. $C \to (C_1, C_2)$ where $\text{amount}(C_1) + \text{amount}(C_2) = \text{amount}(C)$ by construction — $\text{amount}(C_2) := \text{amount}(C) - \text{amount}(C_1)$ computed once, never independently re-derived and checked against a second formula that could disagree with the first. C deactivates atomically with C_1, C_2's issuance; a real, fresh pair of ids is required, never reusing an id already in the claim set.

Transfer. proveTransfer$(C, \text{from}, \text{to})$ requires a real signature over $(\text{claimId}, \text{from}, \text{to}, n, \text{derivation})$ verifiable against from's own real public key — n a real, monotonically-tracked nonce per claim, rejecting any replayed proof outright. Deactivation of the source claim, signature verification, and activation of the destination claim under to all occur within the same real, atomic call — a failed verification anywhere in that sequence discards the whole attempt, never leaving a claim deactivated with no successor.

10 Denomination

1 AIWA $= 10^{18}$ base units, always integer. Reward's fractional exponents require floating-point computation internally; the one real boundary where a value becomes part of a balance converts to an exact integer faithful to the float's own bits.

11 Partition and reconciliation

Each domain continues independently through partition; none decrements state for another's unreachability, none awaits permission. Reconciliation is signature, ancestry, and Mirror-observation verification over newly available evidence — never a question of clock authority.

Channel versus data. A transport session (this reference implementation's own manually-bootstrapped WebRTC, or any real link) is inherently temporary — it exists only while both ends are simultaneously reachable, precisely like a real interplanetary link during a real blackout. Once real data has crossed it, that data is verified and stored durably by each side independently; losing the session never loses what already crossed. Re-establishing a channel after a gap is a real, ordinary event, not a failure — this reference implementation's own manual handshake is a placeholder for whatever real transport (DTN, dedicated hardware) a real deployment would use; the reconciliation logic it feeds (§4, §13) is agnostic to which.

11.1 Positioning

Published interplanetary-cryptocurrency proposals generally extend one Earth-anchored consensus chain across the latency gap — DTN transport, timelocks widened to light-time, federated or merge-mined settlement — leaving consensus and issuance untouched. This transfers already-created value under latency; it leaves creation itself dominated by whichever side has more compute (mining from Mars against Earth's hashpower is acknowledged, in that literature, as structurally unprofitable).

AIWA removes value creation from any consensus chain: epoch_D requires no awareness of one. Reconciliation (§4, §13) is additive and informational only. This closes the specific asymmetry above; it does not address real byte transport under latency (reused, not reinvented, and deliberately pluggable — `sync-protocol.js`'s own real synchronization logic is transport-agnostic by construction, shared identically by a manual WebRTC handshake and an automatic, Nostr-relay-assisted one, with room for a real DTN or dedicated-hardware transport later without touching this logic at all) nor an exchange rate between economies that grew apart.

12 Explicit non-claims

Not solved: the human-identity oracle, physical-location verification, absolute global time, Byzantine agreement without assumptions, detection of every coalition of identities under one real actor. A coalition can produce internally consistent history at real cost. The claim is narrower: fabricated identities cannot fabricate authenticated history *for free*.

12.1 Scalability — real, unaddressed limits

Cross-domain, this scales well by construction — no consensus, no shared bottleneck; verified structurally, not merely asserted (§1). *Within* a single domain, three real costs grow unboundedly and are not yet addressed:

Local storage. At VDF_ITERATIONS = 12000 (∼280ms/epoch, §6), a continuously-running domain produces on the order of 10^8 real progression events per real year — roughly 28GB at a real, conservative ∼250 bytes/event, for one domain alone.

Wallet materialization, now incremental. `rematerialize()`'s own wallet update — the real, VDF-verification-heavy part — now applies only genuinely new events (tracked by the reference application's own `coveredEventIds`) on top of already-materialized state, instead of replaying from scratch. Verified concretely: applying only new events on top of existing state produces byte-identical results to a full replay, for real, causally-independent domains.

Mirror, now incremental on both real sync and real boot. The identical, real fix applied to wallet materialization above, verified the same way (byte-identical to a full replay). `state-snapshot.js`'s own real snapshot — previously wallet-only — now covers Mirror too, saved and loaded together, keyed by the identical real `coveredEventIds`. A real, older, wallet-only snapshot is recognized explicitly (never silently misread as an empty Mirror state) and triggers one real, final full Mirror catch-up before the new, combined snapshot takes over — verified concretely, including a real bigint round trip and the real legacy-format detection itself.

Identity-cost, still a full replay. Cheaper per-event (no VDF verification), but still O(total real history) on every real sync and every real boot — real, remaining, unaddressed cost.

Unbounded full-sync payload. `sync-protocol.js`'s own real full-sync sends the entire real event list as one message on connect — for a domain with a real, large history, this payload grows without bound, with no real, tested ceiling on what a real transport can carry.

None of these are addressed yet — real, open engineering work, not a solved problem being merely under-documented.

13 Causal Tick

A domain's own epoch_D (§5) is unconditional and requires zero external observers. Causal Tick is a complementary, externally-corroborated position — useful for reconciliation and for flagging drift between self-report and observation, never a substitute.

$$\hat{\theta}_X = \text{median}_w\big(\{(\text{obs}_i(X), w_i)\}\big), \qquad w_i = b_i \ (§8)$$

concretely, sort estimates by value; walk the cumulative weight; return the first value where $\sum_{j \le i} w_j \ge \frac{1}{2}\sum_j w_j$. This crossing-point definition — not an average of the two central values — is what makes

the $\sum w_i/2$ robustness bound below exact, not approximate: the returned value is always one a real majority of weight has voted at or below.

One estimate per real observer — their own most-recent, highest-resolving observation of X — never one per historical commitment referencing the same fact; an earlier version double-counted replayed observations, closed here. $[\min_i \text{obs}_i, \max_i \text{obs}_i]$ is reported alongside $\hat{\theta}_X$, not discarded. Robust while adversarial weight stays below $\sum w_i/2$ — proof-of-stake's own security shape, applied to causal position.

$\hat{\theta}_X = \perp$ with zero observers — honest absence, not inconsistency.

Domain invariance. For identical evidence, $\hat{\theta}_X$ is identical regardless of which domain computes it or via which path evidence arrived — a consequence of event ids already being content-addressed (§3): identical causal content produces an identical id independent of transport.

Rejected alternative. A structural diversity measure (distinct branches of new causal content) was considered in place of $w_i = b_i$ and rejected: indistinguishable from a well-resourced actor fabricating many distinct, never-replayed branches — verified concretely (three genuine sources and three sybil-fabricated sources produce identical Shannon entropy). Real committed capital remains the only weight source that closes this.

Never a correction. $\hat{\theta}_X$ is reported, never applied to D's own earned value. Reducing a slower domain's real value toward a faster peer's would punish genuine work under weaker hardware (§6); inflating a reconnecting domain's value toward a peer's would credit elapsed idle time and is directly exploitable by deliberate isolate-then-reconnect cycles. Neither is implemented.

13.1 Hardware roots (optional)

The evidence interface functions on software primitives alone. A domain may optionally strengthen independence assurance with physically-provisioned hardware roots; hardware never computes $\hat{\theta}_X$, never scales w_i, never becomes required input. Zero versus several hardware attestations yield an identical $\hat{\theta}_X$ from identical Mirror evidence — only a separate, informational count differs.

Attestation. A two-hop signature chain: an origin domain signs issuance of a hardware root; the root's own key signs its binding to a target domain. Both independently verified. ≥ 2 distinct, independently-issued roots required — one root moves trust to a single unit, never establishes a minimum.

Stated limit. No cryptographic protocol verifies from software alone that a root's key lives on genuinely non-clonable hardware. What is real: a traceable signature chain from a known origin, reducing trust surface from *any of many sybils* to *the origin's own issuance process, or one physical unit* — a real reduction, not an absolute guarantee.

Identity is the keypair, never a device: loss of hardware is real but never identity loss, restored unchanged from a saved key or passphrase (§8) on any reachable device. This closes device loss, not the case this document is otherwise built around — a domain with no reachable device or channel at all, for a real, extended period. No device, old or new, has anything to reconnect to in that case, regardless of key; hardware roots strengthen assurance for a domain that *is* reachable, and do not substitute for reachability during a real partition.

14 Relative rate, without a clock

A domain's own epoch_D is already an unfakeable marker of real, sequential work (§6). A real, signed statement — "at my own epoch e_O, I observed target X at their own epoch e_X" — uses this marker for something new, and consults no clock, no timestamp, no declared duration, at any point, by any party.

$$w = (\text{observer}, e_O, \text{target}, e_X, \text{sourceEventId})$$

From two such real, successive, valid witnesses w_1, w_2 by the same observer about the same target:

$$\rho = \frac{e_{X,2} - e_{X,1}}{e_{O,2} - e_{O,1}}$$

a purely structural ratio. **This is not underdetermined**, though a single snapshot of raw epoch counts would be: with no external reference, a single comparison cannot distinguish "ran longer" from "ran faster." A second, later observation removes the ambiguity — the absolute interval between w_1 and w_2 is never known or needed; only the real, verified deltas are.

Aggregate. Many real (observer, target) pairs, each weighted by the observer's own real committed capital (§8) — the identical security shape already used for $\hat{\theta}_X$ (§13): a weighted median, robust while adversarial weight stays below half the total real weight contributing an estimate. Verified concretely: ten real, independently-speeded domains recover their real relative rates exactly; a funded but minority-weight adversary reporting a fabricated ratio does not move the aggregate.

Purely informational, by the same principle as §13: this never feeds back into what any domain may claim, never bounds or adjusts epoch_D for anyone. It is offered as a real, emergent statistical regularity — never a synchronization primitive, never a shared clock, never an input any domain's own security depends on. It is a measure of relative rate between a pair, never itself a claim of convergence toward a network-wide homogeneous cadence; whether real-world hardware happens to cluster or spread is an empirical fact about physical reality, neither caused nor established by this mechanism.

Real interface. Mirror's own "Relative rate" card offers a real "Witness now" button per already-observed domain — building and publishing a real, signed witness at this domain's own current epoch. Once two real witnesses exist for the same real target, the real ratio is computed and shown directly, using the identical `computeRelativeRate` above — never a separate, UI-only approximation.

14.1 Composition is unsafe without a freshness bound

$\rho_{AB} \cdot \rho_{BC} = \rho_{AC}$ holds mathematically, but composing two independently-measured real ratios is unsafe if the intermediate domain's own real rate can drift between the two measurements — genuinely, with no forged signature required. Verified concretely: a real, honest change in B's own rate between measuring ρ_{AB} and ρ_{BC} produced a $2\times$ error in the naively composed result. A malicious B, who freely chooses when it measures the next link, can exploit this without forging anything.

Bound. `composeRelativeRates` requires the real, verified epoch gap on B's own progression — between B's epoch as last referenced in the $A \to B$ pair and B's own epoch when it began the $B \to C$ pair — to stay within a caller-supplied `maxFreshnessGap`, refusing composition outright otherwise. Still no clock: the bound is expressed entirely in B's own already-verified epoch count.

A mitigation, not a closure. A tighter bound reduces the window for drift or adversarial timing, verified concretely (a 5-epoch gap composed exactly; a 9900-epoch gap did not), but does not eliminate it — genuine drift can occur within any bound, and the intermediate domain still freely chooses when it measures within that bound. A caller should treat a wide accepted gap as real, reduced confidence, never as equivalent to a short, direct measurement.

15 Generous transfer — deterministic, never chance

A donor may optionally, voluntarily attach a real, additional, conditional bonus to an ordinary transfer (§9) — payable only if a specific, real, future progression event of the *recipient's own* chain meets a public threshold. Structurally identical to mining's own real property: the outcome is a pure function of public data, unpredictable only because one real input (a real, sequential VDF output) genuinely does not exist yet. No randomness anywhere; not a lottery under the conventional consideration/chance/prize test, since the chance element is replaced by real, unforgeable sequential work, exactly as with proof-of-work.

$$c = (\text{contractId}, \text{baseTransferId}, \text{to}, \text{bonusClaimId}, \text{bonusAmount}, \text{thresholdBits}), \quad \text{signed by the donor}$$

$$h = \text{SHA-256}(\text{id}(c) \,\|\, \text{vdfOutput}(e_q)) \qquad \text{win} \iff h \text{ has} \geq \text{thresholdBits leading zero bits}$$

where e_q is a real progression event of to's own chain with $\text{id}(c)$ among its real parents, and e_q's own VDF proof independently re-verified (§6) against to's own real prior output — never trusted from the event's shape alone. Progression's own strict sequentiality (§5) means at most one real e_q can ever exist

for a given real epoch, verified end to end against the real protocol: a real second attempt at an alternate "same epoch" is genuinely rejected (`expected epoch N+1, got N`).

Two real vulnerabilities were found and closed before this was ever shipped, both worth stating plainly rather than only fixing silently. First: an earlier construction combined c with e_q's own full content-addressed id rather than its raw VDF output — since an event's id also depends on cheaply-variable fields (which extra parents are attached), a recipient could privately construct many candidate events reusing the identical, already-computed real VDF output, check each one's outcome, and broadcast only a favorable one, without ever redoing real work. Verified concretely: five candidates, three "wins," using one real VDF computation. Second: without independently re-verifying e_q's own VDF proof, a fabricated vdfOutput — never really, sequentially computed at all — could be ground for a favorable hash directly. Both close for the identical, real reason: the outcome now depends only on values that genuinely require real, sequential, unshortcuttable work to produce or vary — never on a field cheap to change.

No fabricated value. bonusAmount is real, already-owned AIWA the donor irrevocably signed away in advance from their own real bonusClaimId — never created from nothing, never touching any other domain's funds, never a shared pool (§9's own Conservation applies unchanged).

Enforceable by anyone. Resolution requires only real, already-public data — c's own signature, e_q's own real, independently-verifiable content — so the recipient, or any third party, can construct and submit it; the donor's cooperation is never required after the fact, and cannot be withheld.

Real interface. A fourth, real tab — "Give" — sends a real, ordinary transfer alongside an optional, real, at-risk amount; pending offers addressed to this domain, and their real, resolved outcomes, are shown automatically. No new server or infrastructure: the same real progression loop that already runs (§5) also includes any real, pending offer as an extra parent of this domain's own next epoch, resolving it the identical way described above.

Donor-side outcomes. A real win produces a real, findable `contract-payout` event; a real loss produces nothing at all, so it can only ever be *inferred* — never assumed from elapsed time — by having actually received (synced) the real recipient's own progression event that included the offer as a real parent, with no matching payout following. `generous-send-scan.js` — pure, no browser dependency, directly tested — implements this scan; the "Give" tab's own "What you've sent" card surfaces it, with a plain note that a real "no win" can only appear once real reconciliation (§4) has actually reached this domain.

15.1 Contract identity and composability

This is deliberately built as an external contract, never a modification to the core protocol above — the same real, existing primitives (transfer, progression, VDF verification) are only ever *consumed*, never altered. A domain's own address is a direct cryptographic proof (§2); encoding a contract tag into it risks real confusion about which real destination a transfer actually targets, so this contract's own CONTRACT_ID = `aiwa-generous-transfer-v1` is instead part of c's own signed content (above), never mangled into an address. A later, composing contract references this same real, public id explicitly in its own events (e.g. its own real field wrapsContractId = CONTRACT_ID) rather than adopting any address-mangling scheme — "upstream" (other contracts building on this one) and "downstream" (users sending through it directly) both resolve to the identical, real, public string.

15.2 A generic payout mechanism — real, spendable AIWA, without a per-contract core change

For a contract's own conditional outcome to move real, spendable AIWA at all, some real code must eventually apply a real state transition to Conservation's own claim ledger — the one part of this contract genuinely outside its own file. An initial version hardcoded this contract's own verification directly into `wallet.js`; found, during design, not to scale — every future contract wanting the identical real capability would need its own hardcoded case added to the core, contradicting §15's own external-contract principle.

$e_{\text{payout}} = \{\text{type} : \texttt{contract-payout}, \text{contractId}, \text{claimId}, \text{from}, \text{to}, \text{nonce}, \text{timestamp}, \text{signerPubkey}, \text{signature}, \ldots\}$

`wallet.js` instead exposes one real, generic extension point: a real `contractVerifiers` map — {contractId $\mapsto$

verifyPayout} — supplied by the application, never wallet.js's own source. On a real `contract-payout` event, wallet.js independently verifies only what every such contract shares (the pre-signed transfer's own real signature, identical to an ordinary `transfer`), then delegates entirely to the registered contract's own `verifyPayout(payload)` for everything contract-specific. A real, honest `null` from the verifier, or a returned shape not matching the real, submitted event's own fields, rejects outright — nothing is trusted by default, and an unregistered contractId is refused, never silently ignored or treated as valid. `wallet.js`'s own source needs no further change for a new contract; only the application's own registry grows.

16 Publishing a contract, content-addressed

A plain string identifier (§15.1) carries no cryptographic anchor by itself — nothing stops a second, different implementation from claiming the identical name. `contract-registry.js` closes this using the identical, real content-addressing already used for every other event (§3.1): a contract's own real, complete source is embedded — never only its hash — in an ordinary `contract-spec` event published into the same DAG everything else lives in, so the real code is genuinely recoverable by anyone who receives this event, not merely fingerprint-verifiable against a copy kept elsewhere.

$$\text{sourceHash} = \text{SHA-256}(\text{sourceCode}) \qquad e_{\text{spec}} = \text{addEvent}([], \{\text{name}, \text{version}, \text{sourceCode}, \text{sourceHash}, \text{description}\})$$

Once e_{spec} is received by other domains (Mirror, §4, exactly like any other real event), it is immutable in the identical, real sense every event already is here — not a new guarantee, simply what publishing an event has always meant. A single altered character in the real source produces a genuinely different real hash, verified concretely. sourceHash is kept alongside the full body for cheap comparison without re-hashing every time.

No canonical registry, deliberately. This never enforces that exactly one implementation may claim a given name — multiple, real, competing contracts can coexist, each with its own real, verifiable identity; wallets and users choose which to trust, exactly as no single chain enforces one true implementation of a common token standard.

Real interface. The "Give" tab's own "Publish a contract" card exposes this directly — name, version, complete source, description, then a real, immutable, content-addressed id. `scanContractSpecs` lists every real, already-known contract-spec this domain has received. Real transport (§11.1's own P2P sync, unchanged) already carries a `contract-spec` event exactly like any other — publishing was always narrower than transport; now the interface reaches it too.

16.1 A real, asymmetric trust direction for interchain verification (unexplored, not built)

Every real event here is verifiable using only standard, non-proprietary primitives (Ed25519 signatures, SHA-256 content addressing) — never a proprietary format only this project's own software understands. This means a genuinely different trust direction than a typical bridge: a bridge usually requires the receiving chain to trust an oracle or validator set's own claim about what happened elsewhere. Verifying one of *this* project's real events requires trusting no organization at all — only the same, already-public, standard algorithms most chains already implement natively. An external chain wanting to verify a real AIWA event would build its own adapter, entirely on its own terms, using only already-public primitives — never a dependency this protocol introduces or must maintain. This is stated as a real, reasoned architectural property this design already has — not a built feature; no adapter exists yet, and verifying an event's authenticity remains distinct from any external chain's own decision about what to do with that information.

Reference implementation

Concept	File
Identity	`public/core/domain-id.js`
Event DAG	`public/core/event-dag.js`
Sequential proof (§6)	`public/core/vdf.js`, `public/core/wesolowski-vdf.js`, `public/core/bigint-math.js`
Progression (§5)	`public/core/progression.js`
Accrual formula (§7)	`public/core/reward.js`
Accrual position, t/A	`public/core/accrual.js`
Genesis Commitment (§8)	`public/core/identity-cost.js`, `public/core/solana-wallet.js`
Churn profitability check	`public/core/churn-analysis.js` — parameter-specific, not a general guarantee
Cross-runtime interoperability	`interop/rust-vdf/` — independent Rust, byte-identical to JS output
Conservation (§9)	`public/core/conservation.js`
Denomination (§10)	`public/core/units.js`
Mirror (§4)	`public/core/mirror.js`
Causal Tick (§13)	`public/core/causal-tick.js`, `public/core/weighted-median.js`
Hardware roots (§13.1)	`public/core/hardware-attestation.js`
Relative rate, no clock (§14)	`public/core/relative-rate.js`, `public/app/relative-rate-scan.js` — purely informational, weighted like §13
Generous transfer, deterministic (§15)	`public/core/generous-transfer.js`, `public/app/generous-send-scan.js` — never chance, verified end-to-end against the real progression protocol, cross-runtime verified (Rust)
Contract publishing, content-addressed (§16)	`public/core/contract-registry.js`
Live peer-to-peer sync	`public/core/p2p-signaling.js`, `public/app/sync-protocol.js`, `public/app/p2p-connection.js`, `public/app/trystero-connection.js`
Coherent composition	`public/core/wallet.js`

301 tests; every case but the cross-runtime check runs unconditionally, which skips (never fails) without a Rust toolchain. Security-relevant cases are named as such in their own files.